MICROSAR Classic OS HAL

Technical Reference

ARM Cortex-M

Version 1.22.00

Author	Vector Informatik GmbH
Status	Released

Document Information

History

Author	Date	Version	Remarks
virleh, visbpz	2020-03-24	1.0.0	First released version.
visdri, virrlu	2020-04-30	1.1.0	Change family name from “IMX8” to “i.MX 8x” Added chapter Exception Context Manipulation.
visbpz	2020-05-13	1.2.0	Updated HSI for Traveo II Family
visbpz visto	2020-07-14	1.3.0	Updated HSI for i.Mx 8 family HSI refined.
visdri	2020-08-10	1.4.0	Document i.MX RT1171: Manuals, HSI
visdri	2020-09-02	1.5.0	Update i.MX RT1171 HSI
visto	2020-12-08	1.6.0	Add HSI for S32K2TV derivative
virsmn	2020-12-23	1.7.0	Removed author identity
visbpz	2021-03-24	1.8.0	Updated the integration chapter
vismun	2021-04-27	1.9.0	Updated S32K2TV/S32K3xx HSI
vismun	2021-05-27	1.10.0	Support for GCC NXP compiler
vismun	2021-06-21	1.11.0	Multicore support for S32K3xx
vismun	2021-07-12	1.12.0	Generic Cortex-M33 support
vismun	2021-08-02	1.13.0	Updated configuration of X-Signal Support for ARMv8 MPU (PMSAv8)
vismun	2021-09-13	1.14.0	Updated Exception context manipulation
visbpz	2022-02-07	1.15.0	Support for LLVM TI compiler
visbpz	2022-03-03	1.16.0	Support for LLVM Hightec compiler
virmid	2022-03-14	1.17.0	Support for Microchip compiler
virmid	2022-04-25	1.18.0	Support for SAF85xx derivatives
vismun	2022-06-03	1.19.0	Updated S32x and generic Cortex-M33 Updated Wind River Diab compiler version Multicore support for i.MX RT1171
vismun	2022-06-30	1.20.00	Support for S32G3
virrlu, vismun	2022-09-28	1.21.00	Support for S32K39x, S32K38x Changed product name to MICROSAR Classic
vimmoi, xchen	2023-03-29	1.22.00	Updated S32K3xx Reference Manuals. Added hints for SMI-540. Added platform restrictions for i.MX RT1171 MPU regions.

Reference Documents

No.	Source	Title	Version
[1]	Vector	MICROSAR Classic OS Technical Reference	See delivery information
[2]	ARM Limited	ARM v7-M Architecture Reference Manual	
[3]	ARM Limited	ARM v8-M Architecture Reference Manual	
[4]	Vector	MICROSAR Classic SafetyManual	See delivery information



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Introduction 8
1.1 Hardware Overview 8

2 Functional Description 10
2.1 Unhandled Interrupts 10
2.2 Unhandled Exceptions 10
2.3 PSR Register Handling 10
2.4 Exception Context Manipulation 11
2.4.1 Exception context reading 11
2.4.2 Exception context modification 14

3 Integration 15
3.1 Hardware Software Interface 15
3.1.1 Generic Cortex-M 15
3.1.1.1 Context Registers 15
3.1.1.2 Core Registers 15
3.1.1.3 Core MPU Registers 16
3.1.1.4 INTC Registers 16
3.1.2 ATSAMv7x Family 17
3.1.2.1 Context Registers 17
3.1.2.2 Core Registers 17
3.1.2.3 Core MPU Registers 17
3.1.2.4 INTC Registers 17
3.1.3 S32K14x Family 18
3.1.3.1 Context Registers 18
3.1.3.2 Core Registers 18
3.1.3.3 System MPU Registers 18
3.1.3.4 INTC Registers 18
3.1.3.5 Timer Module Registers 18
3.1.4 TDA2x 19
3.1.4.1 Context Registers 19
3.1.4.2 Core Registers 19
3.1.4.3 INTC Registers 19
3.1.4.4 IPU Registers 19
3.1.4.5 Spinlock Registers 19
3.1.5 Traveo II Family 20
3.1.5.1 Context Registers 20
3.1.5.2 Core Registers 20
3.1.5.3 Core MPU Registers 20

3.1.5.4 INTC Registers 20
3.1.5.5 IPC Registers 20
3.1.5.6 Timer Module Registers 21
3.1.6 i.MX 8 Family 22
3.1.6.1 Context Registers 22
3.1.6.2 Core Registers 22
3.1.6.3 Core MPU Registers 22
3.1.6.4 INTC Registers 22
3.1.6.5 TPM Registers 22
3.1.6.6 MU Registers 23
3.1.7 i.MX RT1171 Family 24
3.1.7.1 Context Registers 24
3.1.7.2 Core Registers 24
3.1.7.3 Core MPU Registers 24
3.1.7.4 INTC Registers 24
3.1.7.5 GPT Timer registers 24
3.1.7.6 MU Registers 25
3.1.7.7 IOMUX Controller LPSR GPR Registers 25
3.1.7.8 System Reset Controller Registers 25
3.1.8 S32x Family (S32G, S32R and S32V) 26
3.1.8.1 Context Registers 26
3.1.8.2 Core Registers 26
3.1.8.3 Core MPU Registers 26
3.1.8.4 INTC Registers 26
3.1.8.5 Spinlock 26
3.1.8.6 Mode Entry Module 27
3.1.8.7 Reset Generation Module 27
3.1.8.8 STM Registers 27
3.1.9 S32K3xx Family 28
3.1.9.1 Context Registers 28
3.1.9.2 Core Registers 28
3.1.9.3 Core MPU Registers 28
3.1.9.4 INTC Registers 28
3.1.9.5 Spinlock 28
3.1.9.6 Mode Entry Module 29
3.1.9.7 STM Registers 29
3.1.10 Generic Cortex-M33 30
3.1.10.1 Context Registers 30
3.1.10.2 Core Registers 30
3.1.10.3 Core MPU Registers 31
3.1.10.4 INTC Registers 31

3.1.11 SAF85xx Family 32
3.1.11.1 Context Registers 32
3.1.11.2 Core Registers 32
3.1.11.3 Core MPU Registers 32
3.1.11.4 INTC Registers 32
3.1.11.5 STM Registers 32
3.2 Configuration of X-Signal 33
3.2.1 X-Signal ISR Interrupt Sources 33
3.3 Configuration of Interrupt Mapping 33
3.3.1 Traveo II 33
3.3.2 i.MX 8x 34
3.4 Category 1 ISRs before StartOS 34
3.5 Note on SMI-540 34

4 Platform Restrictions 35
4.1 Arm Cortex-M33 Architecture 35
4.2 Exception Handlers 35
4.3 Memory 35
4.4 Stack usage measurement 37
4.5 Section Symbols 37
4.6 Floating Point Context 37

5 Derivative Restrictions 38
5.1.1 General 38
5.1.2 S32K14x Family 38
5.1.3 S32x (S32G, S32R and S32V) and S32K3xx Family 39
5.1.4 TDA2x Family 39
5.1.5 i.MX 8x Family 39
5.1.6 i.MX RT1171 Family 39

6 Contact 40

Tables

Table 1-1 Supported ARM Hardware 9
Table 1-2 Supported ARM compilers 9
Table 2-1 Unhandled exceptions 10
Table 3-1 Global interrupt flag / Interrupt priority registers 34
Table 4-1 Supported ARMv8-M configurations.....34

1 Introduction

This document describes platform specific details about the usage and functions of “MICROSAR Classic OS” on the ARM Cortex-M derivative family. General information about “MICROSAR Classic OS” can be found in [1].

1.1 Hardware Overview

The following table summarizes information about MICROSAR Classic OS. It gives detailed information about the derivatives and supported compilers of the ARM Cortex-M derivative family. As very important information the documentations of the hardware manufacturers are listed. MICROSAR Classic OS is based upon these documents in the given version.

Derivative Family	Hardware Manufacturer Document Name	Document Version
i.MX8 x	i.MX 8DualXPlus/8QuadXPlus Application Processor Reference Manual, i.MX 8QuadMax Applications Processor Reference Manual, i.MX 8DualXLite Applications Processor Reference Manual	Rev. B, 5/2018, Rev. F, 10/2019, Rev. B, 03/2020
ATSAMV7x	SAM v70 Datasheet	Rev.11297D, 06/2016
S32K14x	NXP/Freescale - S32K14x Series Reference Manual - Supports S32K142, S32K144, S32K146, and S32K148	Rev. 3, 03/2017
Generic_CortexM	ARMv7-M Architecture Reference Manual	v.E.b 12/2015
Generic_CortexM0	ARMv6-M Architecture Reference Manual	ARM DDI 0419D 19 May 2017
TDA2x	TDA2x Technical Reference Manual	SPRUI29D, November 2015
CYT2Bx	Traveo II Automotive Body Controller Entry Family Architecture Technical Reference Manual	Rev.*B, February 2018
CYT4Bx	Traveo II Automotive Body Controller High Family Architecture Technical Reference Manual: 002-24401 CYT4BF Data sheet (32-bit Arm Cortex-M7 Microcontroller Traveo II Family), CYT3BB/4BB Data sheet (32-bit Arm Cortex-M7 Microcontroller Traveo II Family)	Rev.*C, November 2019, Rev.F, 15 November 2019, Rev.C, 15 November 2019
S32x	S32V344 Reference Manual S32G Reference Manual S32R45 Reference Manual S32G3 Reference Manual	Rev.1 Draft M, Juni 2018 S32G274ARM, Rev.1, Nov 2019 S32R45RM Rev. 1, Jan 2020 S32G3RM Rev. 1, Nov 2021

NXP S32K2TV	S32K2xx Reference Manual	S32K2xxRM Rev.1 Draft N, 07/2019
NXP S32K3xx	S32K3 Reference Manual	S32K3xxRM Rev.5, 09/2022 S32K396RM Rev. 1.1, 09/2022
NXP i.MX RT1171	i.MX RT1170 Processor Reference Manual	Document Number: IMXRT1170RM : Rev. H, 10/2020
Generic_CortexM33	ARMv8-M Architecture Reference Manual	ARM DDI 0553B.i 02 October 2019
SAF85xx	SAF85xx Digital Reference Manual	Rev. 1.8, 28 January 2022

Table 1-1 Supported ARM Hardware

Green Hills (GHS)	2013.5.4 and later (for ARMv7-M), 2018.5.4 and later (for ARMv8-M)
IAR	7.70.x and later (for ARMv7-M), 9.10.x (for ARMv8-M)
TI	15.12.3.LTS
ARM CC	5.06u1 and later
GCC Linaro Distribution	gcc-linaro-7.1.1-2017.08-i686-mingw32_arm-eabi
Tasking	v6.2r2p2
GCC HighTec	v4.6.5.1
GCC NXP	v6.3.1
Wind River Diab (LLVM)	7.0.2.1 and later
Texas Instruments (LLVM)	1.3.0.LTS and later
HighTec (LLVM)	8.0.0 and later
Microchip (xc32)	v2.40

Table 1-2 Supported ARM compilers

2 Functional Description

2.1 Unhandled Interrupts

In case of an unhandled interrupt request the application can obtain the source number of the triggering interrupt request by the OS API Os_GetUnhandledIrq() (see [1] for details). On the ARM Cortex-M derivative family, the 32-bit value returned by the API represents the source of the interrupt as found in the exception table, if it is a Core local interrupt without mapping. In case the interrupt controller supports mapping and the interrupt is mapped, the system interrupt source is provided.

2.2 Unhandled Exceptions

In case of an unhandled exception the application can obtain the source number of the triggering exception by the OS API Os_GetUnhandledExc() (see [1] for details). On the ARM Cortex-M derivative family, the 32-bit value returned by the API represents the exception source number. The unhandled exception numbers range from 2 to 14.

Exception Number	Comment
2	NMI (Non Maskable Interrupt) exception is per default unhandled. Can be triggered through hardware or software.
3	HardFault is handled by the OS when memory protection (SC3/SC4) is enabled, otherwise is unhandled.
4	MemManage is handled by the OS when memory protection (SC3/SC4) is enabled is, otherwise is unhandled.
5	BusFault is handled by the OS when memory protection (SC3/SC4) is enabled, otherwise is unhandled.
6	UsageFault is triggered through non-memory related faults.
7-10	Are reserved on the platform and should not happen.
12	DebugMonitor is unhandled.
13	Is reserved on the platform and should not happen.
14	PendSV is not utilized by the OS.

Table 2-1 Unhandled exceptions

2.3 PSR Register Handling

PSR.T bit handling	The Thumb execution state bit is left unchanged during runtime. This bit is controlled by hardware.
PSR.N/Z/C/V handling	The condition code flags bits are left unchanged during runtime. Thes bits are controlled by hardware.

2.4 Exception Context Manipulation

Manipulating the exception context using Os_SetExceptionContext is not supported (see [1] for details). But for debugging purposes, the Cortex-M derivative family supports the feature to read the exception context when a hardware exception occurs using the OS API Os_GetExceptionContext.

2.4.1 Exception context reading

When an unhandled exception (see Table 2-1 Unhandled exceptions) occurs, a structure variable representing the exception context is filled at the beginning of exception handling. The elements of this structure variable are either copied directly from registers or from the “stack frame” pushed automatically to the stack on exception entry. A reference to this structure variable is returned to the caller when the API OS_GetExceptionContext is called.

The context structure variable is filled differently depending on:

- whether the stack frame was successfully pushed to the stack or not, and on
- whether floating point context saving is supported or not, and on
- whether MPU is supported / configured or not.

The following table summarizes the values given to the different elements of the context structure variable filled immediately on entry to an unhandled exception.

	Context structure variable element	Valid stack frame	Invalid stack frame	Floating-point element content when FP context saving is not supported	Thread stack MPU element when memory protection is not supported	Comments
Extended Stack Frame	ValidStackFrame	0x1	0x0			(0)
	R0	From register	From register			(1)
	R1	From register	From register			(1)
	R2	From register	From register			(1)
	R3	From stack	0xDEAD			(2), (3)
	R12	From stack	0xDEAD			(2), (3)
	LR	From stack	0xBADF			(2), (4)
	ReturnAddress	From stack	0xBADF			(2), (4)
	xPSR	From stack	0xBADF			(2), (4)
	FP lower registers: S0 .. S15	From stack	0xBADF	0xDEAD		(2), (4), (5)
	FPSCR	From stack	0xBADF	0xDEAD		(2), (4), (5)
Special purpose registers	Original_SP	From stack	From register			(6), (1)
	PRIMASK	From register	From register			(1)
	BASEPRI	From register	From register			(1)
	FAULTMASK	From register	From register			(1)
	CONTROL	From register	From register			(1)
	R4 .. R11	From register	From register			(1)
	FP upper registers: S16 .. S31	From register	From register	0xDEAD		(1), (5)
Thread stack registers	StackMpuRegionStartInfo	From register	From register		0xDEAD	(1), (7)
	StackMpuRegionEndInfo	From register	From register		0xDEAD	(1), (7)

(0) The element ValidStackFrame indicates whether hardware errors occurred when pushing the stack frame to the stack. Two types of errors are checked

- Failure on a hardware save of context on exception entry, because of an MPU access violation.
- Failure on a hardware save of context on exception entry due to a bus error.

(1) “From register” means that a register content is copied directly to the structure variable element. Depending on the element, the register is either

- a general-purpose register or
- a special purpose register or
- a floating-point extension register or
- a MPU register.

(2) “From stack” means that the content of the corresponding element is copied from the “stack frame” pushed to the stack on exception entry. In case floating-point context saving is supported, the stack frame is called “extended stack frame” (see [2] for details).

(3) “0xDEAD” here indicates that

- it wasn’t possible to copy the element content directly from a register because the register content was already modified by the SW implementation in the exception handler
- and that it was not possible to copy it from the stack because the stack content is invalid.

(4) “0xBADF” here indicates that this element, which on exception entry could only be retrieved from the stack, couldn’t be retrieved from the stack because the “stack frame” content is invalid.

(5) “0xDEAD” is written to this element when the floating-point context saving is not supported.

(6) “Original_SP” element refers to

- the stack pointer content of the interrupted function in case the stack frame was pushed correctly to the stack. This element is retrieved from the stack in this case.
- the current SP register content in case the stack frame couldn’t be pushed correctly to the stack. In this case, the current SP points to a position in memory that has a negative offset to the interrupted function SP. The offset is equal to the stack frame size. This is because, when the exception occurs, the hardware, firstly, updates the SP register to the new expected position, then tries to push the stack frame to the stack.

  Please note that the mentioned offset contains or not additional 4-bytes due to stack alignment (see “[2], B1.5.6 Exception entry behavior, PushStack()” for details).

(7) This element holds information about the MPU region corresponding to the current thread stack e.g. thread stack start address, end address / size, ...

The content of this element depends on whether the MPU is supported / configured or not and on the type of the supported MPU.

- The following table gives the content of the element depending on the type of the supported MPU.

Context structure variable element	Cortex-v7M core MPU	Cortex-v8M core MPU	NXP system MPU
StackMpuRegionStartInfo	MPU_RBAR register content (see [2] for details)	MPU_RBAR register content (see [3] for details)	RGDx_WORD0 register content which is the MPU region corresponding to the current thread stack region start address.
StackMpuRegionEndInfo	MPU_RASR register content (see [2] for details)	MPU_RLAR register content (see [3] for details)	RGDx_WORD1 register content which is the MPU region corresponding to the current thread stack region end address.

- “0xDEAD” is written to this element when memory protection is not supported.

2.4.2 Exception context modification

The Cortex-M derivative family does not support the feature to modify the exception context using Os_SetExceptionContext (see [1] for details).

3 Integration

3.1 Hardware Software Interface

The following chapter describes the Hardware-Software Interface of the ARM Cortex-M derivative family.

There are registers which are always under control of the OS and shall never be altered by application software.

On the other hand, there are registers which are only under supervision of the OS if certain aspects have been configured. The following chapters will list under which configuration aspects such registers are handled exclusively by the OS.

3.1.1 Generic Cortex-M

3.1.1.1 Context Registers

Register	Handled by OS	Altering by application SW
R4-R11	Always	Allowed
PC	Always	Not allowed
LR	Always	Not allowed
SP	Always	Not allowed
PSR	Always	Not allowed
CONTROL	Always	Not allowed
BASEPRI	Always	Not allowed*

3.1.1.2 Core Registers

Register	Handled by OS	Altering by application SW
MPIDR	Always	Allowed
PRIMASK	Always	Not allowed*
FAULTMASK	Always	Not allowed
CCR	Always	Not allowed
SHPR1	Always	Not allowed
SHPR2	Always	Not allowed
SHPR3	Always	Not allowed
SHCSR	Always	Not allowed
SYST_CSR	Always	Not allowed
SYST_RVR	Always	Not allowed
SYST_CVR	Always	Not allowed
SYST_CALIB	Always	Not allowed

*3.4

3.1.1.3 Core MPU Registers

These registers are exclusively used by the OS, if memory protection has been configured (SC3 or SC4 systems).

In systems without configured memory protection (SC1 or SC2) application software may alter these registers.

- MPU_TYPE
- MPU_CTRL
- MPU_RNR
- MPU_RBAR
- MPU_RASR

3.1.1.4 INTC Registers

Register	Handled by OS	Altering by application SW
SHPR	Always	Not allowed
NVIC_ISER	Bits within these registers which are related to category 2 ISRs are handled by the OS.	The application may access bits (within these registers) which are related to category 0 or category 1 ISRs. For enable/disable interrupt source or to set or clear pending bits.
NVIC_ICER		
NVIC_ISPR		
NVIC_ICPR		
NVIC_AIBR	Always	Not allowed
NVIC_IPR	Always	Not allowed
ICSR	Always	Not allowed
VTOR	Always	Not allowed
STIR	Always	Not allowed

3.1.2 ATSAMv7x Family

3.1.2.1 Context Registers

See chapter 3.1.1.1

3.1.2.2 Core Registers

See chapter 3.1.1.2

3.1.2.3 Core MPU Registers

See chapter 3.1.1.3

3.1.2.4 INTC Registers

See chapter 3.1.1.4

3.1.3 S32K14x Family

3.1.3.1 Context Registers

See chapter 3.1.1.1

3.1.3.2 Core Registers

See chapter 3.1.1.2

3.1.3.3 System MPU Registers

These registers are exclusively used by the OS, if memory protection has been configured (SC3 or SC4 systems).

In systems without configured memory protection (SC1 or SC2) application software may alter these registers.

- SMPU_CESR
- SMPU_RGDn_WORD0
- SMPU_RGDn_WORD1
- SMPU_RGDn_WORD2
- SMPU_RGDn_WORD3
- SMPU_RGDAAC0

3.1.3.4 INTC Registers

See chapter 3.1.1.4

3.1.3.5 Timer Module Registers

If an FTM timer is configured to drive an OS counter of type “HARDWARE” the corresponding timer registers are exclusively used by the OS.

The application may use FTM timer if it is not configured within the OS.

- FTM_SC
- FTM_CNT
- FTM_CnSC
- FTM_CnV

3.1.4 TDA2x

3.1.4.1 Context Registers

See chapter 3.1.1.1

3.1.4.2 Core Registers

See chapter 3.1.1.2

3.1.4.3 INTC Registers

See chapter 3.1.1.4

3.1.4.4 IPU Registers

- CORTEXM4_CTRL_REG (this register exclusively used by the OS in a multicore environment)
- CORTEXM4_RW_PID1 (this register read by the OS in a multicore environment, but it must be initialized by application software)

3.1.4.5 Spinlock Registers

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- SPINLOCK_SYSCONFIG
- SPINLOCK_SYSTATUS
- SPINLOCK_LOCK_REG_0

3.1.5 Traveo II Family

3.1.5.1 Context Registers

See chapter 3.1.1.1

3.1.5.2 Core Registers

See chapter 3.1.1.2

3.1.5.3 Core MPU Registers

See chapter 3.1.1.3

3.1.5.4 INTC Registers

Beside the registers describe in chapter 3.1.1.4 there are additional registers related to interrupt handling.
These registers are exclusively used by the OS and may never be altered by application software.

- CPUSS_CM4_SYSTEM_INT_CTL (Body Entry)
- CPUSS_CM4_INTm_STATUS (Body Entry, m = 0…7)
- CPUSS_CM7_0_SYSTEM_INT_CTL (Body High)
- CPUSS_CM7_1_SYSTEM_INT_CTL (Body High)
- CPUSS_CM7_0_INTm_STATUS (Body High, m = 0…7)
- CPUSS_CM7_1_INTm_STATUS (Body High, m = 0…7)

3.1.5.5 IPC Registers

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- IPC_STRUCT7_ACQUIRE (Body High, reserved for Spinlocks)
- IPC_STRUCT7_RELEASE (Body High, reserved for Spinlocks)
- IPC_STRUCTm_NOTIFY (Body High, m = 0…6)
- IPC_STRUCTm_INTR (Body High, m = 0…6)
- IPC_STRUCTm_INTR_MASK (Body High, m = 0…6)
- CPUSS_IDENTITIY

3.1.5.6 Timer Module Registers

If a timer from timer module is configured to drive an OS counter of type “HARDWARE” the corresponding timer registers are exclusively used by the OS.

The application may use a timer if it is not configured within the OS.

- TCPWM0_GRP2_CNTm_CTRL (Reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_COUNTER (Reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_CC0 (Reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_TR_CMD (Reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_INTR (Reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_INTR_MASK (Reserved for HRT, m= 0…4)

3.1.6 i.MX 8 Family

3.1.6.1 Context Registers

See chapter 3.1.1.1

3.1.6.2 Core Registers

See chapter 3.1.1.2

3.1.6.3 Core MPU Registers

See chapter 3.1.1.3

3.1.6.4 INTC Registers

Beside the registers describe in chapter 3.1.1.4 there are additional registers related to interrupt handling.
These registers are exclusively used by the OS and may never be altered by application software.

- IRQSTEER_CHAN0CTL
- IRQSTEER_CHAN0_MINTDIS
- IRQSTEER_(CHAN0_MASK0 – CHAN0_MASKn)
- IRQSTEER_(CHAN0_STATUS0 – CHAN0_STATUSn)

3.1.6.5 TPM Registers

If a TPM timer is configured to drive an OS counter of type “HARDWARE” the corresponding timer registers are exclusively used by the OS.

The application may use TPM timer if it is not configured within the OS.

- TPM_SC (Reserved for HRT)
- TPM_CONF (Reserved for HRT)
- TPM_QDCTRL (Reserved for HRT)
- TPM_C0SC (Reserved for HRT)
- TPM_C0V (Reserved for HRT)
- TPM_CNT (Reserved for HRT)
- TPM_MOD (Reserved for HRT)

3.1.6.6 MU Registers

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- MUA_ACR (MU0_A0, Reserved for Xsignals)
- MUB_BCR (MU0_B, Reserved for Xsignals)
- MUB_BSR (MU0_B, Reserved for Xsignals)

3.1.7 i.MX RT1171 Family

3.1.7.1 Context Registers

See chapter 3.1.1.1

3.1.7.2 Core Registers

See chapter 3.1.1.2

3.1.7.3 Core MPU Registers

See chapter 3.1.1.3

3.1.7.4 INTC Registers

See chapter: INTC Registers

3.1.7.5 GPT Timer registers

i.MX RT1171 provides 6 GPT timers. If one of these timers is configured to drive an OS counter of type “HARDWARE” the corresponding timer registers are exclusively used by the OS except GPTCR and GPT_PR registers.

The fields CLKSRC and EN_24M in the Control Register GPT_CR are to be configured by the user of MICROSAR Classic OS who is expected to choose/enable the GPT clock source. The field CLKSRC must be written before the OS initialization i.e. before that the “Os_Init()” function is called. The same applies to the field EN_24M when the external 24MHz oscillator is selected as a clock source.

GPT_PR register can be written by the user to configure the GPT pre-scalers of the input clock. MICROSAR Classic OS does not write this register. Hence, if these pre-scalers are not written by the user, the reset value “0” of this register is maintained (i.e. no input clock frequency division is performed).

Register	Handled by OS	Modification by application SW
GPT_ CR (Control Register)	Yes: all fields except CLKSRC and EN_24M	The fields CLKSRC (bits 6-8) and EN_24M (bit 10) are to be configured by the user to select/enable the clock source for this timer. The other fields shall not be modified.
GPT_ PR (Prescaler Register)	No	Yes
GPT_ SR (Status Register)	Yes	Not allowed
GPT_IR (Interrupt Register)	Yes	Not allowed
GPT_ OCR1 (Output Compare Register 1)	Yes	Not allowed
GPT_ OCR2 (Output Compare Register 2)	Yes	Not allowed
GPT_ OCR3 (Output Compare Register 3)	Yes	Not allowed
GPT_ ICR1 (Input Capture Register 1)	Yes	Not allowed
GPT_ ICR2 (Input Capture Register 2)	Yes	Not allowed
GPT_ CNT (Counter Register)	Yes	Not allowed

3.1.7.6 MU Registers

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured) for Xsignals.

Application software may use them in a single core system (only one core is configured as Autosar Core).

- MU_ACR
- MU_ASR
- MU_BCR
- MU_BSR

3.1.7.7 IOMUX Controller LPSR GPR Registers

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured) to set the vector table base address of Cortex-M4.

Application software may use them in a single core system (only one core is configured as Autosar Core).

- IOMUXC_LPSR_GPR_GPR0
- IOMUXC_LPSR_GPR_GPR1

3.1.7.8 System Reset Controller Registers

The register is exclusively used by the OS within a multicore system (more than one Autosar Core is configured) to release Cortex-M4 core reset.

Application software may use it in a single core system (only one core is configured as Autosar Core).

- SRC_SCR

3.1.8 S32x Family (S32G, S32R and S32V)

3.1.8.1 Context Registers

See chapter 3.1.1.1

3.1.8.2 Core Registers

See chapter 3.1.1.2

3.1.8.3 Core MPU Registers

See chapter 3.1.1.3

3.1.8.4 INTC Registers

Beside the registers describe in chapter 3.1.1.4 there are additional registers related to interrupt handling.
These registers are exclusively used by the OS and may never be altered by application software.

- MSCM_IRSPRC
- MSCM_IRCPISR
- MSCM_IRCPIGR
- MSCM_CPUXNUM

3.1.8.5 Spinlock

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- XRDC_ HWCFG1
- SEMA42_GATE0
- SEMA42_RSTGT

3.1.8.6 Mode Entry Module

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- MC_ME_CTL_KEY
- MC_ME_PRTN0_COREn_ADDR , n = [1,3]
- MC_ME_PRTN0_COREn_PCONF , n = [1,3]
- MC_ME_PRTN0_COREn_PUPD , n = [1,3]
- MC_ME_PRTN0_COREn_STAT , n = [1,3]

3.1.8.7 Reset Generation Module

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- RGM_PRST
- RGM_PSTAT

3.1.8.8 STM Registers

If a STM timer is configured to drive an OS counter of type “HARDWARE” the corresponding timer registers are exclusively used by the OS.

The application may use STM timer if it is not configured within the OS.

- STM_TS_CR
- STM_TS_CNT
- STM_TS_CCR0
- STM_TS_CIR0
- STM_TS_CMP0

3.1.9 S32K3xx Family

3.1.9.1 Context Registers

See chapter 3.1.1.1

3.1.9.2 Core Registers

See chapter 3.1.1.2

3.1.9.3 Core MPU Registers

See chapter 3.1.1.3

3.1.9.4 INTC Registers

Beside the registers describe in chapter 3.1.1.4 there are additional registers related to interrupt handling.
These registers are exclusively used by the OS and may never be altered by application software.

- MSCM_IRSPRC
- MSCM_IRCPISR
- MSCM_IRCPIGR
- MSCM_CPUXNUM

3.1.9.5 Spinlock

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- XRDC_ HWCFG1
- SEMA42_GATE0
- SEMA42_RSTGT

3.1.9.6 Mode Entry Module

These registers are exclusively used by the OS within a multicore system (more than one Autosar Core is configured).

Application software may use them in a single core system (only one core is configured as Autosar Core).

- MC_ME_CTL_KEY
- MC_ME_PRTN0_CORE1_ADDR
- MC_ME_PRTN0_CORE1_PCONF
- MC_ME_PRTN0_CORE1_PUPD
- MC_ME_PRTN0_CORE1_STAT
- MC_ME_PRTN0_CORE4_ADDR
- MC_ME_PRTN0_CORE4_PCONF
- MC_ME_PRTN0_CORE4_PUPD
- MC_ME_PRTN0_CORE4_STAT

3.1.9.7 STM Registers

If a STM timer is configured to drive an OS counter of type “HARDWARE” the corresponding timer registers are exclusively used by the OS.

The application may use STM timer if it is not configured within the OS.

- STM_TS_CR
- STM_TS_CNT
- STM_TS_CCR0
- STM_TS_CIR0
- STM_TS_CMP0

3.1.10 Generic Cortex-M33

3.1.10.1 Context Registers

Register	Handled by OS	Altering by application SW
R4-R11	Always	Allowed
PC	Always	Not allowed
LR	Always	Not allowed
SP	Always	Not allowed
PSR	Always	Not allowed
CONTROL	Always	Not allowed
BASEPRI	Always	Not allowed*

3.1.10.2 Core Registers

Register	Handled by OS	Altering by application SW
PRIMASK	Always	Not allowed*
FAULTMASK	Always	Not allowed
SHPR1	Always	Not allowed
SHPR2	Always	Not allowed
SHPR3	Always	Not allowed
SHCSR	Always	Not allowed
SYST_CSR	Always	Not allowed
SYST_RVR	Always	Not allowed
SYST_CVR	Always	Not allowed
SYST_CALIB	Always	Not allowed

* refer to 3.4

3.1.10.3 Core MPU Registers

Register	Handled by OS	Usage by application SW
PRBAR	If memory protection has been configured (SC3 or SC4 systems)	Allowed in SC1 and SC2 systems with privileged access
PRLAR		
PRSELR		

3.1.10.4 INTC Registers

Register	Handled by OS	Altering by application SW
SHPR	Always	Not allowed
NVIC_ISER	Bits within these registers which are related to category 2 ISRs are handled by the OS.	The application may access bits (within these registers) which are related to category 0 or category 1 ISRs. For enable/disable interrupt source or to set or clear pending bits.
NVIC_ICER		
NVIC_ISPR		
NVIC_ICPR		
NVIC_AIBR	Always	Not allowed
NVIC_IPR	Always	Not allowed
ICSR	Always	Not allowed
VTOR	Always	Not allowed
STIR	Always	Not allowed

3.1.11 SAF85xx Family

3.1.11.1 Context Registers

See chapter 3.1.1.1

3.1.11.2 Core Registers

See chapter 3.1.1.2

3.1.11.3 Core MPU Registers

See chapter 3.1.1.3

3.1.11.4 INTC Registers

Beside the registers describe in chapter 3.1.1.4 there are additional registers related to interrupt handling.
These registers are exclusively used by the OS and may never be altered by application software.

- MSCM_IRSPRC
- MSCM_IRCPISR
- MSCM_IRCPIGR
- MSCM_CPUXNUM

3.1.11.5 STM Registers

If a STM timer is configured to drive an OS counter of type “HARDWARE” the corresponding timer registers are exclusively used by the OS.

The application may use STM timer if it is not configured within the OS.

- STM_TS_CR
- STM_TS_CNT
- STM_TS_CCR0
- STM_TS_CIR0
- STM_TS_CMP0

3.2 Configuration of X-Signal

Logical Priority	A low number for OsIsrInterruptPriority attribute means a high logical priority
X-Signal ISR Interrupt Priority	Aside from the priority rules for the X-Signal receiver ISR described in [1] the OsIsrInterruptPriority can be chosen freely.

3.2.1 X-Signal ISR Interrupt Sources

TDA2x	Interrupt source 19 has to be used for the X-Signal ISRs.
S32x*	Interrupt source 17 has to be used for the X-Signal ISRs.
S32K3xx	Interrupt source 16 has to be used for the X-Signal ISRs.
TraveoII–BH**	Interrupt sources 0…6 have to be used for the X-Signal ISR. (IPC IRQs)
i.MX 8x	Interrupt source 45 has to be used for the X-Signal ISRs. (MU0_B0 IRQ)
i.MX RT1171	Interrupt source 134 has to be used for the X-Signal ISRs. (MU IRQ)

3.3 Configuration of Interrupt Mapping

3.3.1 Traveo II

Supported Interrupt Type	CoreExternalIRQ0..CoreExternalIRQ7
ISR Category	Any
Supported on derivative families	Traveo II

The Interrupt Mapping feature on Traveo II is used to map Core external peripheral interrupt sources to one of the 8 CoreExternalIRQ channels.

After mapping an interrupt to any CoreExternalIRQ channel, the interrupt is added to the generated System Interrupt Vector Table. An interrupt not mapped to any type will be treated as a normal core local interrupt source.

* S32x comprises S32G, S32R and S32V platforms.
** TraveoII–BH comprises CYT4Bx and CYT4Dx platforms.

3.3.2 i.MX 8x

Supported Interrupt Type	CoreExternalIRQ0..CoreExternalIRQ7
ISR Category	Any
Supported on derivative families	i.MX 8x

The Interrupt Mapping feature on i.MX 8x is used to map Core external peripheral interrupt sources to one of the 8 CoreExternalIRQ channels.There are limitations on which interrupts can be mapped to which sources.

After mapping an interrupt to any CoreExternalIRQ channel, the interrupt is added to the generated System Interrupt Vector Table. An interrupt not mapped to any type will be treated as a normal core local interrupt source.

3.4 Category 1 ISRs before StartOS

Refer to [1] in order to find out the steps necessary to activate and serve Category 1 ISRs before the OS has been started. Table 3-1 Global interrupt flag / Interrupt priority registers shows the registers that are **only allowed** to be written for servicing Category 1 ISRs before StartOs. After the OS has been started the rules of the Hardware Software Interface (HSI) apply, for details refer to 3.1.

Platform	Register	Altering by application SW
Generic Cortex-M / All derivative families	BASEPRI	Allowed
	PRIMASK	Allowed

Table 3-1 Global interrupt flag / Interrupt priority registers

3.5 Note on SMI-540

The OS safety manual [4] defines SMI-540. This safety manual item states that the user of MICROSAR Classic Safe shall verify that the user software does not contain any system call instructions.

Within the ARM Cortex-M architecture this is the “svc” instruction. User code must not include this instruction.

A possible verification method might be to review the user code for (inline) assembly statements, pragmas or intrinsic functions containing the “svc” instruction.

4 Platform Restrictions

4.1 Arm Cortex-M33 Architecture

ARMv8-M architecture security extension is optional, therefore the architecture supports different configurations. Table 4.1 demonstrates which configurations are supported by MICROSAR Classic OS implementation.

Without TrustZone technology	Supported
With TrustZone technology and additionally TrustZone feature is disabled on the hardware i.e. out of reset the core runs in the non-secure state	Supported
With TrustZone technology and additionally TrustZone feature is enabled on the hardware i.e. out of reset the core runs in the secure state	Supported; if and only if the main function starts in non-secure state and the code never returns to secure state. Meaning the OS and all its threads run in non-secure state

Table 4-1 Supported ARMv8-M configurations

4.2 Exception Handlers

- The exception handler for Supervisor Call is implemented by the OS
- The exception handler for Systick is implemented by the OS

4.3 Memory

Caution with ARMv7 MPU (PMSAv7)
The MPU configuration must not contain the regions with the number higher than the number of available MPU regions minus 2. One region with the highest number is always reserved for the stack protection.

E.g. if 16 regions are available, only the region numbers from 0 to 14 (inclusive) are allowed.

Caution with ARMv7 MPU (PMSAv7)
Due to ARM MPU hardware restriction the sizes of ARM MPU regions and stack sizes must be configured with power of 2 values. An ARM CPU may be equiped with a non-ARM MPU, in that case the aforementioned limitation does not generally hold and the documentation of the MPU should be consulted.

Caution with ARMv8 MPU (PMSAv8)
ARMv8 MPU regions must not overlap.

Caution with ARMv8 MPU (PMSAv8)
MICROSAR Classic OS reserves memory region numbers 0, 1 and 2 to reprogram stack memory regions dynamically during context switches. The user, therefore, must neither configure any memory region(s) for the stacks, nor use any of the aforementioned memory region numbers.

Caution with ARMv8 MPU (PMSAv8)
MICROSAR Classic OS supports all possible configurations for memory region attributes. Nevertheless, ARMv8 architecture restricts, during the run time, number of memory region attributes up to eight different attributes. The user, therefore, must not configure more than seven different memory region attributes excluding (TYPE_NORMAL_MEMORY_INNER_NON_CACHEABLE and TYPE_NORMAL_MEMORY_OUTER_NON_CACHEABLE) which is always generated by the OS.

Caution with ARMv8 MPU (PMSAv8)
ARMv8 architecture supports device memory and normal memory as two main categories for memory region attributes.

- For device memory type: only one memory region attribute should be specified by the user, for instance TYPE_DEVICE_nGnRE_MEMORY.
- For normal memory type: the user should select two memory region attributes to define INNER and OUTER policies, for example TYPE_NORMAL_MEMORY_INNER_WRITEBACK_TRANSIENT_WRITEALLOCATE and TYPE_NORMAL_MEMORY_OUTER_WRITEBACK_TRANSIENT_WRITEALLOCATE

Caution with i.MX RT1171 MPU
Due to the fact that the i.MX RT1171 has 16 MPU regions on the Cortex-M7, 8 MPU regions on the Cortex-M4, and the static generation of the stack memory region number in the OS implementation, MICROSAR Classic OS uses only up to 8 MPU regions for the Cortex-M7.

4.4 Stack usage measurement



Caution with GCC compiler
If the feature “Stack Usage Measurement” is activated and one of the OS stacks (managed by the OS) is applied before calling Os_Init, then the optimization option tree-loop-distribute-patterns needs to be disabled.

4.5 Section Symbols



Note
When using the ARM compiler, the OS generator will not generate section start and stop symbols. However, the ARM linker will provide region-related symbols with special patterns (e.g. Image$$region_name$$Base or Load$$region_name$$Base), which can be used to configure start and end addresses of MPU regions, peripheral regions or access check region objects. Detailed information about the region-related symbols can be found in the user guide of the ARM compiler.

4.6 Floating Point Context



Note
When setting OsFpuUsage to ALL or INDIVIDUAL additional memory and runtime is required for FPU context handling. This affects the following derivative families:

> ARM Cortex-M



Caution
MICROSAR Classic OS expects that the Lazy Context Save feature is disabled at startup (FPCCR.LSPEN bit is set to 0).

5 Derivative Restrictions

5.1.1 General



Caution with Interrupt Priorities
Due to ARM v7M architecture restriction on how to handle interrupts, the interrupt priority of any Category 2 interrupt (including CounterIsr) that can cause scheduling should be lower than all other Category 2 interrupts. This is needed to avoid unexpected delays in interrupt excecution and interrupt overruns.



Caution with exception vector table address

To avoid memory violations directly after boot phase, the exception vector table has to be linked correctly (derivative specific) by the user linker script.

e.g. ATSAMv7 Derivative expects that section OS_INTVEC_CORE<Core Id>_CODE is linked to address 0x00400000 (first available internal flash address).



Note for GCC compiler
The interrupt vector tables are in the sections with the name “<Core_Name>_VectorTable_Section”. These sections need to be 128 bytes aligned.

5.1.2 S32K14x Family



Caution
Region 0 of the System MPU is reserved for debugging functionality and could not be written by the core. This region is not available for user configuration.

Caution
Due to FTM module hardware restriction (only 16 Bit count and compare registers), the prescaler of the timer module is set to 32 by software. This is needed to achieve an adequate timer resolution. This needs to be considered when configuring the parameter SecondsPerTick for an HRT OS-Counter.

5.1.3 S32x (S32G, S32R and S32V) and S32K3xx Family

Caution
MICROSAR Classic OS expects that Hardware Configuration 1 register (XRDC_HWCFG1) provides the correct domain number of the requesting bus master (Processor cores). The domain number must be unique for each core in range of [0-15].

5.1.4 TDA2x Family

Caution
MICROSAR Classic OS expects that register CORTEXM4_RW_PID1 is initialized with the physical core id before starting.

5.1.5 i.MX 8x Family

Caution
MICROSAR Classic OS expects that on Multicore derivatives the Tightly Coupled Memory is enabled and the start of TCM (address 0x20000000) is initialized with the physical core id before starting.

5.1.6 i.MX RT1171 Family

Caution
On multicore derivatives MICROSAR Classic OS expects Cortex-M7 core to be the auto-start core, which is the default hardware configuration. Additionally, MICROSAR Classic OS uses CPU identification register (CPUID) to distinguish each core. Cortex-M7 and Cortex-M4 are identified as Core 0 and Core 1 respectively.

6 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2023 Vector Informatik GmbH